Temporal Edge Weighting for Graph Collaborative Filtering*

Minje Kim and Sang-Min Choi

Gyeongsang National University, Jinju, Republic of Korea
{alswp6597, jerassi}@gnu.ac.kr

Abstract

Graph collaborative filtering is widely used for learning user and item representations from user–item interaction graphs. However, many existing backbones rely on a largely static interaction graph, where observed interactions are treated uniformly regardless of their timestamps. In this paper, we propose a lightweight plug-and-play temporal edge weighting scheme that directly incorporates temporal recency into the propagation graph. The proposed method assigns time-aware weights to user–item edges and can be instantiated with personalized, global, and hybrid recency strategies, without modifying the original backbone architectures. We apply the proposed scheme to representative graph collaborative filtering backbones, including NGCF, LightGCN, and LR-GCCF, and evaluate it on Yelp2018 and ML-1M.Experimental results show that the effect of temporal edge weighting is dataset- and backbone-dependent: it yields more consistent gains on ML-1M, while its effect on Yelp2018 is less consistent, suggesting that recency is beneficial only when it aligns with the preference dynamics of the target domain. The proposed scheme introduces no additional learnable parameters, and the personalized and global variants generally remain close to the original backbones in runtime, whereas the hybrid variant can incur additional overhead. These findings suggest that edge-level temporal recency modeling can serve as a practical backbone-preserving option for graph-based recommendation.

1 Introduction

Graph collaborative filtering remains a strong and widely used approach for learning from user–item interactions [1, 15]. By leveraging neighborhood aggregation on the user–item graph, it captures higher-order collaborative signals beyond directly observed edges while maintaining relatively simple architectures. Representative models such as NGCF [16], LightGCN [9], and LR-GCCF [4] differ in their aggregation and update mechanisms, but they all follow the common principle of learning user and item representations through iterative neighborhood interaction on the user–item graph. Owing to their effectiveness, simplicity, and flexibility, these methods remain important backbones and baselines in recommendation research.

Despite their strong empirical performance, many graph collaborative filtering backbones still rely on a largely static interaction graph, where observed interactions are treated uniformly once they are included in the graph [14, 17]. However, user–item interactions are inherently temporal. In some domains, recent interactions may better reflect a user's current interests, whereas older interactions may become less informative as preferences evolve over time. At

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 22, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

the same time, temporal recency is not universally beneficial, since older interactions can still encode stable long-term preferences, especially in domains where user tastes change slowly. Therefore, a key question is not simply whether temporal information should be used, but how recency can be incorporated into graph propagation while preserving the usefulness of long-term collaborative signals.

To address this limitation, we propose a lightweight plug-and-play temporal edge weighting scheme for graph-based recommendation. Our method assigns time-aware weights to user–item edges so that more informative recent interactions contribute more strongly during graph propagation, while less relevant older interactions are appropriately discounted. The proposed approach can be incorporated into multiple representative graph collaborative filtering backbones, including NGCF, LightGCN, and LR-GCCF, with minimal modification to their original architectures. Accordingly, the main goal of this work is to provide an architecture-preserving way to inject temporal recency into standard graph collaborative filtering models.

By directly injecting temporal recency into the propagation graph, the proposed method preserves the original backbone architectures while providing a lightweight way to adjust edge importance during message passing. Empirical results on Yelp2018 and ML-1M show that temporal edge weighting can improve recommendation performance, but its effectiveness depends on the dataset characteristics and the underlying graph backbone. In particular, temporal weighting yields more consistent gains on ML-1M, while its effect on Yelp2018 is more model-dependent. These findings suggest that explicit edge-level recency modeling can serve as a practical backbone-preserving enhancement for graph-based recommendation.

- We propose a lightweight temporal edge weighting scheme that directly incorporates temporal recency into the user–item propagation graph, instantiated with personalized, global, and hybrid strategies, without modifying the original graph collaborative filtering backbones.
- We empirically show that the effect of temporal edge weighting is dataset- and backbone-dependent. The proposed weighting strategies yield more consistent gains on ML-1M, while the results on Yelp2018 reveal the importance of preserving long-term preference signals.
- We analyze the runtime behavior of the proposed variants and show that personalized and global weighting introduce no additional learnable parameters and generally remain close to the original backbones in training time, whereas hybrid weighting involves a clearer computational trade-off.

2 Related Work

2.1 Graph Collaborative Filtering Methods

Graph collaborative filtering has established a strong line of research in recommendation by modeling users and items as a bipartite graph and learning embeddings through graph propagation. Due to their strong effectiveness and simplicity, such models have become representative backbones in diverse recommendation research. In multi-behavior recommendation [11], recent models often employ LightGCN-based encoders to learn behavior-specific user and item representations from multiple interaction graphs [6, 5]. In multimodal recommendation [20], graph-based backbones such as LightGCN and NGCF are commonly used as the collaborative propagation module while incorporating visual and textual signals [21, 13]. A similar trend

also appears in social recommendation [12], where graph-based social recommenders serve as core backbones for modeling interaction and social relations [10, 18]. These studies illustrate that graph collaborative filtering backbones are frequently reused as core propagation modules across different recommendation settings. This motivates our focus on modifying the propagation graph itself rather than redesigning the backbone architectures.

2.2 Temporal Modeling in Graph-based Recommendation

Prior work has incorporated temporal information into graph-based recommendation in several ways. Temporal collaborative filtering with graph convolutional neural networks proposes a temporal collaborative filtering method that leverages GNNs for representation learning and RNNs for modeling temporal dynamics [3]. DGSR [19] models sequential recommendation through a dynamic graph structure and a Dynamic Graph Recommendation Network that captures user–item interactions with time and order information. TGSRec [7] further introduces a continuous-time bipartite graph and a Temporal Collaborative Transformer to unify sequential patterns and temporal collaborative signals. More recently, TAGCF [22] incorporates time factors and attention into graph collaborative filtering to model temporal context during embedding propagation.

While these studies demonstrate the importance of temporal information in graph-based recommendation, they often introduce additional temporal modules, dynamic graph construction procedures, attention mechanisms, or sequential modeling components. In contrast, our objective is different: we focus on whether temporal recency can be injected into existing graph collaborative filtering backbones without changing their architectures. The proposed method does not introduce a specialized temporal encoder or additional learnable parameters, but instead replaces the binary interaction graph with a time-aware weighted propagation graph. Therefore, our contribution lies in an architecture-preserving temporal reweighting formulation for standard GCF backbones, rather than in designing a new temporal recommendation architecture.

3 Preliminaries

3.1 Problem Definition

Let $\mathcal{U}$ and $\mathcal{I}$ denote the user and item sets, respectively, and let

$$\mathcal{D} = \{(u, i, t_{ui}) \mid u \in \mathcal{U},\ i \in \mathcal{I}\} \tag{1}$$

be the set of timestamped user–item interactions. From the observed interaction history, we construct a user–item bipartite graph

$$\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E}), \tag{2}$$

where $\mathcal{E}$ denotes the observed user–item edges in the graph.

Given $\mathcal{G}$, the task is to rank candidate items for each user. For a user u, let $\mathcal{I}_u$ denote the set of items observed in the interaction history of u, and define the candidate set as

$$\mathcal{C}_u = \mathcal{I} \setminus \mathcal{I}_u. \tag{3}$$

The model assigns a relevance score $f(u, j \mid \mathcal{G})$ to each candidate item $j \in \mathcal{C}_u$ and produces a personalized ranking accordingly.

3.2 Graph Collaborative Filtering Backbones

Let

$$\mathbf{H}^{(0)} = \begin{bmatrix} \mathbf{E}_U^{(0)} \\ \mathbf{E}_I^{(0)} \end{bmatrix} \tag{4}$$

denote the stacked initial user and item embeddings. Let $\hat{\mathbf{A}}$ denote the normalized user–item interaction graph used for message passing. Although NGCF, LightGCN, and LR-GCCF differ in how they aggregate and combine graph-based signals, all three models fundamentally learn representations through repeated operations on $\hat{\mathbf{A}}$. Since our method modifies this graph to incorporate temporal information, we briefly summarize below only the adjacency-related components of these representative graph collaborative filtering backbones.

- **NGCF** [16] updates node representations by combining graph-based neighborhood aggregation with a bi-interaction term. At layer l, it first computes neighborhood messages from the normalized interaction graph and then applies nonlinear transformations to both the aggregated message and the element-wise interaction between the current and aggregated representations:

$$\mathbf{H}^{(l+1)} = \sigma(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}_{\text{gc}}^{(l)}) + \sigma\Big((\mathbf{H}^{(l)} \odot \hat{\mathbf{A}}\mathbf{H}^{(l)})\mathbf{W}_{\text{bi}}^{(l)}\Big). \tag{5}$$

  The final representation is obtained by concatenating the embeddings from all layers.

- **LightGCN** [9] adopts a simplified linear graph aggregation scheme in which the normalized interaction graph directly propagates representations across layers:

$$\mathbf{H}^{(l+1)} = \hat{\mathbf{A}}\mathbf{H}^{(l)}. \tag{6}$$

  Its final embedding is the average of the embeddings from all layers:

$$\mathbf{Z}_{\text{LightGCN}} = \frac{1}{L+1}\sum_{l=0}^{L}\mathbf{H}^{(l)}. \tag{7}$$

- **LR-GCCF** [4] also builds user and item representations through repeated multiplication with the normalized interaction graph:

$$\mathbf{H}^{(l+1)} = \hat{\mathbf{A}}\mathbf{H}^{(l)}. \tag{8}$$

  Unlike LightGCN, it uses layer-wise concatenation to form the final representation:

$$\mathbf{Z}_{\text{LRGCCF}} = \mathbf{H}^{(0)} \,\|\, \mathbf{H}^{(1)} \,\|\, \cdots \,\|\, \mathbf{H}^{(L)}. \tag{9}$$

4 Method

Motivated by the shared reliance of graph collaborative filtering backbones on the normalized interaction graph [9, 4, 16], we enhance recommendation by revising the graph used for message passing rather than changing the backbone architectures themselves. Specifically, we replace the original static interaction graph with a time-aware weighted graph whose edge weights are determined by interaction timestamps, thereby incorporating temporal recency into the representation learning process while preserving the original backbone structure.

The observed user–item interactions are represented by the binary interaction matrix

$$\mathbf{R}_{ui} = \begin{cases} 1, & (u, i, t_{ui}) \in \mathcal{D}, \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

From $\mathbf{R}$, we define the corresponding bipartite adjacency matrix as

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{0} \end{bmatrix} + \delta_{\text{loop}} \mathbf{I}, \tag{11}$$

where δ_{loop} indicates whether self-loops are used. We then normalize $\mathbf{A}$ according to the backbone-specific rule to obtain the propagation graph $\hat{\mathbf{A}}$ used for message passing.

4.1 Personalized Temporal Weighting

We first define temporal recency relative to each user's own interaction history. Let t_{ui} denote the timestamp of interaction (u, i), and let t_u^{recent} denote the most recent interaction time of user u in the training set. We then define the personalized time gap as

$$\Delta_{ui}^{\text{P}} = \min\left\{ \max\left(\frac{t_u^{\text{recent}} - t_{ui}}{s}, 0 \right), c \right\}, \tag{12}$$

where $s > 0$ is a time scaling factor and $c > 0$ is a clipping threshold. Based on this gap, the personalized edge weight is defined as

$$w_{ui}^{\text{P}} = \epsilon + (1 - \epsilon) \exp(-\lambda_{\text{P}} \Delta_{ui}^{\text{P}}), \tag{13}$$

where $\epsilon \in (0, 1)$ is a minimum edge-weight floor and $\lambda_{\text{P}} > 0$ is the decay coefficient.

Instead of the original binary interaction matrix $\mathbf{R}$, we define a personalized time-weighted interaction matrix $\mathbf{R}^{\text{P}}$ as

$$\mathbf{R}_{ui}^{\text{P}} = \begin{cases} w_{ui}^{\text{P}}, & (u, i, t_{ui}) \in \mathcal{D}, \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

From $\mathbf{R}^{\text{P}}$, we construct the corresponding bipartite adjacency matrix $\mathbf{A}^{\text{P}}$, and then obtain the propagation graph $\hat{\mathbf{A}}^{\text{P}}$ through backbone-specific normalization.

4.2 Global Temporal Weighting

Similar to personalized temporal weighting, we define global temporal weighting based on the most recent observed interaction. Let $t^{\max}$ denote the most recent observed timestamp in the interaction history. The global time gap is defined as

$$\Delta_{ui}^{\text{G}} = \min\left\{ \max\left(\frac{t^{\max} - t_{ui}}{s}, 0 \right), c \right\}, \tag{15}$$

and the corresponding global edge weight is

$$w_{ui}^{\text{G}} = \epsilon + (1 - \epsilon) \exp(-\lambda_{\text{G}} \Delta_{ui}^{\text{G}}), \tag{16}$$

where $\lambda_{\text{G}} > 0$ is the decay coefficient. We then replace the original binary interaction matrix $\mathbf{R}$ with the global time-weighted interaction matrix

$$\mathbf{R}_{ui}^{\text{G}} = \begin{cases} w_{ui}^{\text{G}}, & (u, i, t_{ui}) \in \mathcal{D}, \\ 0, & \text{otherwise,} \end{cases} \tag{17}$$

from which the corresponding bipartite adjacency matrix $\mathbf{A}^{\text{G}}$ and normalized propagation graph $\hat{\mathbf{A}}^{\text{G}}$ are obtained in the same manner as in the personalized case.

Table 1: Statistics of the datasets.

Dataset	# Users	# Items	# Interactions	Sparsity
Yelp2018	77,278	45,639	2,103,896	99.94%
ML-1M	6,041	3,261	998,539	94.93%

4.3 Hybrid Temporal Weighting

Personalized and global temporal weighting reflect two different notions of recency. Personalized temporal weighting is defined relative to a user's own interaction history, whereas global temporal weighting is defined relative to the most recent observed interaction. To incorporate both temporal views, we combine the corresponding normalized propagation graphs during message passing.

Given hidden representations $\mathbf{H}^{(l)}$, we define the hybrid propagation operator as

$$\Phi_{\mathrm{h}}(\mathbf{H}^{(l)}) = \gamma \hat{\mathbf{A}}^{\mathrm{P}} \mathbf{H}^{(l)} + (1-\gamma) \hat{\mathbf{A}}^{\mathrm{G}} \mathbf{H}^{(l)}, \tag{18}$$

where $\gamma \in [0, 1]$ controls the balance between the two temporal views.

Accordingly, personalized temporal weighting uses $\hat{\mathbf{A}}^{\mathrm{P}}$ as the propagation graph, whereas global temporal weighting uses $\hat{\mathbf{A}}^{\mathrm{G}}$. For hybrid temporal weighting, every occurrence of $\hat{\mathbf{A}}\mathbf{H}^{(l)}$ in the backbone is replaced by $\Phi_{\mathrm{h}}(\mathbf{H}^{(l)})$, with all other parts of the backbone preserved. In this way, the proposed temporal weighting can be plugged into multiple graph collaborative filtering backbones with minimal modification.

5 Experiments

5.1 Experimental Settings

5.1.1 Datasets

We conduct experiments on two widely used benchmarks for recommendation: Yelp2018 [2] and ML-1M [8]. We apply 10-core filtering to each dataset. Table 1 summarizes the resulting dataset statistics. These datasets provide different levels of sparsity and preference dynamics. Yelp2018 is a highly sparse local business review dataset, where user preferences over restaurants, venues, and service categories may remain relatively stable over time. In contrast, ML-1M is denser and contains more structured movie rating interactions, where recent interactions may more reliably reflect changes in user interests. This contrast allows us to examine when temporal recency is helpful and when preserving long-term preference signals remains important

5.1.2 Evaluation

We adopt the widely used leave-one-out evaluation protocol under a chronological split. For each user, the most recent interaction is used for testing, the second most recent interaction for validation, and the remaining interactions for training. During evaluation, we rank all candidate items for each user without negative sampling. We report three standard top-K ranking metrics, $Recall@K$, $NDCG@K$, and $MRR@10$, where $K \in \{10, 20\}$. $Recall@K$ measures whether the target item appears in the top-K list, while $NDCG@K$ further accounts for its rank position. $MRR@10$ measures the reciprocal rank of the target item within the top-10 list, giving higher values when the target item is ranked earlier.

Table 2: Performance comparison on Yelp2018. The best results are shown in **bold**, and the second-best is underlined. #Improve denotes the relative improvement of the best-performing proposed variant over the original backbone.

Model	Variant	Recall@10	Recall@20	NDCG@10	NDCG@20	MRR@10	MRR@20
NGCF	Original	**0.0436**	**0.0746**	**0.0213**	**0.0290**	**0.0146**	**0.0166**
	Personalized	0.0388	0.0672	0.0188	0.0259	0.0127	0.0147
	Global	0.0412	0.0704	0.0200	0.0273	0.0136	0.0156
	Hybrid	0.0403	0.0695	0.0196	0.0269	0.0134	0.0154
	Improve (%)	-5.82%	-5.96%	-6.50%	-5.22%	-7.35%	-6.41%
LightGCN	Original	**0.0489**	**0.0830**	**0.0241**	**0.0326**	**0.0166**	**0.0189**
	Personalized	0.0423	0.0728	0.0203	0.0280	0.0138	0.0158
	Global	0.0486	0.0820	0.0239	0.0323	0.0164	0.0187
	Hybrid	0.0461	0.0782	0.0225	0.0306	0.0155	0.0177
	Improve (%)	-0.61%	-1.20%	-0.83%	-0.92%	-1.21%	-1.06%
LR-GCCF	Original	0.0409	0.0693	0.0200	0.0271	0.0137	0.0156
	Personalized	0.0395	0.0676	0.0191	0.0262	0.0130	0.0149
	Global	**0.0431**	**0.0734**	**0.0211**	**0.0287**	**0.0146**	**0.0166**
	Hybrid	0.0415	0.0706	0.0203	0.0276	0.0139	0.0159
	Improve (%)	5.10%	5.58%	5.21%	5.57%	6.16%	6.02%

5.1.3 Implementation Details

We instantiate the proposed temporal weighting on NGCF, LightGCN, and LR-GCCF while preserving their original architectures. The batch size is fixed to 2048, and the latent dimensionality of user and item embeddings is fixed to 64 for all models. For the remaining hyperparameters, we follow the original implementations of each backbone as closely as possible.

For temporal weighting, we set the common scaling and clipping parameters to $s = 86400$ and $c = 365$, respectively, so that raw timestamps are converted into days and excessively large time gaps are clipped within a one-year range. We further use $\epsilon = 0.05$, $\lambda_{\mathrm{P}} = 0.1$, $\lambda_{\mathrm{G}} = 0.1$, and $\gamma = 0.5$. All reported results are averaged over five runs with different random seeds.

5.2 Result Analysis

5.2.1 Overall Performance

Table 2 reports the overall performance on Yelp2018. On this dataset, the effect of temporal weighting is model-dependent. For NGCF and LightGCN, the original models achieve the best performance across all metrics; in contrast, LR-GCCF benefits from temporal weighting, with the global variant achieving the best results across all metrics and improving over the original model by about 5–6%. These results indicate that temporal recency is not universally beneficial. Yelp2018 is highly sparse and corresponds to local business review behavior, where older interactions can still represent stable long-term preferences over restaurants, venues, or service categories. In such a setting, aggressively discounting past interactions may weaken useful collaborative signals, especially for backbones that already rely strongly on neighborhood aggregation. The positive results on LR-GCCF suggest that the effect of temporal weighting also depends on the propagation mechanism of the underlying backbone.

Table 3 presents the overall performance on ML-1M. Compared with Yelp2018, temporal weighting shows more consistently positive effects across the three backbones, although the best

Table 3: Performance comparison on ML-1M. The best results are shown in **bold**, and the second-best is underlined. #Improve denotes the relative improvement of the best-performing proposed variant over the original backbone.

Model	Variant	Recall@10	Recall@20	NDCG@10	NDCG@20	MRR@10	MRR@20
NGCF	Original	0.0716	0.1290	0.0332	0.0476	0.0218	0.0257
	Personalized	0.0726	0.1305	0.0340	0.0485	0.0225	0.0264
	Global	**0.0748**	0.1314	**0.0351**	**0.0493**	**0.0233**	**0.0271**
	Hybrid	0.0723	**0.1322**	0.0335	0.0485	0.0220	0.0260
	Improve (%)	4.27%	2.42%	5.41%	3.44%	6.43%	5.16%
LightGCN	Original	0.0741	0.1338	0.0347	0.0496	0.0229	0.0270
	Personalized	0.0741	**0.1339**	**0.0349**	**0.0499**	**0.0232**	**0.0273**
	Global	**0.0743**	0.1331	0.0347	0.0495	0.0230	0.0269
	Hybrid	0.0742	0.1332	0.0347	0.0495	0.0230	0.0270
	Improve (%)	0.26%	0.07%	0.57%	0.60%	1.29%	1.09%
LR-GCCF	Original	0.0728	0.1306	0.0345	0.0490	0.0231	0.0270
	Personalized	0.0745	**0.1336**	0.0353	**0.0501**	**0.0236**	**0.0276**
	Global	0.0754	0.1325	**0.0355**	0.0498	**0.0236**	0.0275
	Hybrid	**0.0755**	0.1332	0.0353	0.0498	0.0233	0.0272
	Improve (%)	3.57%	2.24%	2.81%	2.19%	2.11%	2.17%

temporal variant still differs by model. For NGCF, the global variant achieves the best performance on most metrics, while the hybrid variant obtains the best Recall@20. For LightGCN, all variants perform very similarly, with only marginal improvements from temporal weighting, indicating that its simplified propagation already provides a stable graph signal on this dataset. For LR-GCCF, temporal weighting consistently improves over the original model, with personalized, global, and hybrid variants each achieving the best result on different metrics. These results indicate that temporal weighting is broadly useful on ML-1M, but the most effective temporal view remains model-dependent.

The different behavior between Yelp2018 and ML-1M can be explained by the interaction between sparsity, domain characteristics, and temporal preference dynamics. Yelp2018 is much sparser than ML-1M, so old interactions may provide important connectivity and collaborative signals in the user–item graph. Moreover, local business preferences can remain stable over time, making strong recency-based discounting less suitable. In contrast, ML-1M is denser and more structured, which makes it less dependent on old edges for graph connectivity. Movie preferences may also exhibit clearer temporal shifts as users consume different genres or styles over time. As a result, temporal edge weighting can emphasize recent preference signals on ML-1M without losing as much useful long-term information. Therefore, the proposed method should be viewed as a domain-dependent enhancement whose effectiveness depends on how well temporal recency aligns with the target dataset.

5.2.2 Training Efficiency

Table 4 reports the average training time per epoch (s/epoch) over five runs for each model variant on Yelp2018 and ML-1M. Since the proposed method reweights existing edges in the interaction graph, it does not introduce additional learnable parameters or increase the number of observed user–item edges. After the weighted adjacency matrix is constructed, the personalized and global variants use a single propagation graph, as in the original backbones. Therefore, their asymptotic message-passing cost remains the same order as the corresponding backbone, i.e., proportional to the number of nonzero edges used in sparse graph propagation.

Table 4: Average training time per epoch (s/epoch) over five runs for each model variant on Yelp2018 and ML-1M. Lower values indicate faster training.

Model	Variant	Yelp2018	ML-1M
NGCF	Original	77.41	**16.02**
	Personalized	81.97	17.49
	Global	**71.09**	16.32
	Hybrid	117.92	28.71
LightGCN	Original	23.99	6.31
	Personalized	50.50	**6.19**
	Global	**22.47**	6.53
	Hybrid	64.47	9.15
LR-GCCF	Original	31.88	**7.27**
	Personalized	54.26	7.43
	Global	**30.62**	7.34
	Hybrid	62.71	34.72

The measured runtime, however, does not always increase monotonically after temporal weighting. In some cases, a weighted variant is slightly faster than the original model. This should not be interpreted as lower computational complexity. Since sparse graph propagation is affected by low-level implementation details such as sparse tensor coalescing, memory access patterns, GPU kernel scheduling, caching behavior, and normal system-level variance, small runtime differences can occur even when the theoretical operation count is similar. In our setting, temporal weighting changes the numerical values and normalization of the propagation graph while largely preserving its sparsity pattern, which can lead to minor variations in low-level execution efficiency.

In contrast, the hybrid variant is consistently slower because it constructs and uses two separate normalized propagation graphs, $\hat{\mathbf{A}}^{\mathrm{P}}$ and $\hat{\mathbf{A}}^{\mathrm{G}}$, during message passing. This approximately doubles the sparse propagation operations associated with the temporal graph component. Therefore, the hybrid variant should be understood as a trade-off between combining two notions of recency and incurring additional runtime overhead. Overall, the personalized and global variants provide the more practical lightweight options, whereas the hybrid variant may be useful only when its accuracy gain justifies the additional computational cost.

6 Conclusion & Future Work

In this paper, we proposed a lightweight temporal edge weighting scheme for graph collaborative filtering. The proposed method injects temporal recency into the user–item propagation graph through time-aware edge weights, without modifying the backbone architecture, scoring function, or learning objective. We instantiated this scheme with personalized, global, and hybrid strategies and applied it to representative graph collaborative filtering backbones. Experiments on Yelp2018 and ML-1M show that temporal edge weighting has dataset- and backbone-dependent effects. The method yields more consistent gains on ML-1M, while the results on Yelp2018 are less consistency, indicating that recency-based weighting is beneficial only when recent interactions better reflect the preference dynamics of the target domain. These results also highlight the importance of preserving long-term preference signals in domains where older interactions remain informative. In terms of efficiency, the personalized and global variants introduce no additional learnable parameters and generally remain close to the original

backbones in training time, whereas the hybrid variant can incur noticeable overhead due to the use of two propagation graphs. Future work will further examine the proposed temporal edge weighting scheme through sensitivity analysis of temporal hyperparameters, comparisons with temporal and sequential recommendation approaches, evaluations on larger and more temporally dynamic datasets, and deeper analysis of the trade-off between temporal recency and long-term preference preservation.

Acknowledgements

Following are results of a study on the "Gyeongsangnam-do Regional Innovation System & Education (RISE)" Project, supported by the Ministry of Education and Gyeongsangnam-do (Project Number: 1-1-1-1-7-14). This research was supported by the Regional Innovation System & Education (RISE) program through the RISE Center, Gyeongsangnam-do, funded by the Ministry of Education (MOE) and the Gyeongsangnam-do Provincial Government, Republic of Korea. (2025-RISE-16-001).

References

[1] Vineeta Anand and Ashish Kumar Maurya. A survey on recommender systems using graph neural network. *ACM Transactions on Information Systems*, 43(1):1–49, 2025.

[2] Nabiha Asghar. Yelp dataset challenge: Review rating prediction. *arXiv preprint arXiv:1605.05362*, 2016.

[3] Esther Rodrigo Bonet, Duc Minh Nguyen, and Nikos Deligiannis. Temporal collaborative filtering with graph convolutional neural networks. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 4736–4742. IEEE, 2021.

[4] Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. Revisiting graph based collaborative filtering: A linear residual graph convolutional network approach. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 27–34, 2020.

[5] Zhiyong Cheng, Jianhua Dong, Fan Liu, Lei Zhu, Xun Yang, and Meng Wang. Disentangled cascaded graph convolution networks for multi-behavior recommendation. *ACM Transactions on Recommender Systems*, 2(4):1–27, 2024.

[6] Zhiyong Cheng, Sai Han, Fan Liu, Lei Zhu, Zan Gao, and Yuxin Peng. Multi-behavior recommendation with cascading graph convolution networks. In *Proceedings of the ACM web conference 2023*, pages 1181–1189, 2023.

[7] Ziwei Fan, Zhiwei Liu, Jiawei Zhang, Yun Xiong, Lei Zheng, and Philip S Yu. Continuous-time sequential recommendation with temporal graph collaborative transformer. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pages 433–442, 2021.

[8] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.

[9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 639–648, 2020.

[10] Wei Jiang, Xinyi Gao, Guandong Xu, Tong Chen, and Hongzhi Yin. Challenging low homophily in social recommendation. In *Proceedings of the ACM Web Conference 2024*, pages 3476–3484, 2024.

[11] Kyungho Kim, Sunwoo Kim, Geon Lee, Jinhong Jung, and Kijung Shin. Multi-behavior recommender systems: a survey. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 435–452. Springer, 2025.

[12] Xiao Li, Li Sun, Mengjie Ling, and Yan Peng. A survey of graph neural network based recommendation in social networks. *Neurocomputing*, 549:126441, 2023.

[13] Kai Ouyang, Chen Tang, Wenhao Zheng, Xiangjin Xie, Xuanji Xiao, Jian Dong, Hai-Tao Zheng, and Zhi Wang. Knowledge soft integration for multimodal recommendation. *arXiv preprint arXiv:2305.07419*, 2023.

[14] Shaowen Peng, Kazunari Sugiyama, and Tsunenori Mine. Svd-gcn: A simplified graph convolution paradigm for recommendation. In *Proceedings of the 31st ACM international conference on information & knowledge management*, pages 1625–1634, 2022.

[15] Tin T Tran, Loc Tan Nguyen, and Nguyen Ngoc Phien. Graph neural networks for collaborative filtering: A survey on ranking prediction. *IEEE Access*, 2026.

[16] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*, pages 165–174, 2019.

[17] Xiang Wang, Hongye Jin, An Zhang, Xiangnan He, Tong Xu, and Tat-Seng Chua. Disentangled graph collaborative filtering. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*, pages 1001–1010, 2020.

[18] Yonghui Yang, Le Wu, Zihan Wang, Zhuangzhuang He, Richang Hong, and Meng Wang. Graph bottlenecked social recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3853–3862, 2024.

[19] Mengqi Zhang, Shu Wu, Xueli Yu, Qiang Liu, and Liang Wang. Dynamic graph neural networks for sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4741–4753, 2022.

[20] Hongyu Zhou, Xin Zhou, Zhiwei Zeng, Lingzi Zhang, and Zhiqi Shen. A comprehensive survey on multimodal recommender systems: Taxonomy, evaluation, and future directions. *arXiv preprint arXiv:2302.04473*, 2023.

[21] Xin Zhou and Zhiqi Shen. A tale of two graphs: Freezing and denoising graph structures for multimodal recommendation. In *Proceedings of the 31st ACM international conference on multimedia*, pages 935–943, 2023.

[22] Xianglin Zuo, Xin He, Tianhao Jia, and Ying Wang. Graph collaborative filtering model combining time factor and attention mechanism. *Journal of Artificial Intelligence Research*, 84, 2025.